

1 План подготовки к собеседованию по Go

Интенсивный курс по ключевым темам для прохождения технического собеседования

Цель: глубокое понимание Go для уверенного прохождения интервью на позицию Go-разработчика уровня PT EDR.

2 1. Базовые типы данных

2.1 Слайсы (Slices)

- slice header: ptr, len, cap — структура, лежащая в основе любого слайса.
- **Аллокация:** механизм append; когда создаётся новый массив, а когда используется существующий (рост ёмкости).
- **Escape analysis:** когда слайс выделяется на стеке, а когда на куче.
- **Передача в функцию:** изменение элементов — разрешено, изменение len/cap — нет.

2.2 Мапы (Maps)

- Внутреннее устройство: hmap, бакеты, хеш-таблица.
- **Итерация:** порядок рандомизирован (и не должен использоваться в логике).
- **Потокобезопасность:** мапы небезопасны для конкурентной записи.
- **sync.Map vs sync.RWMutex + map:** когда что применять.

2.3 Каналы (Channels)

- hchan: устройство канала (буфер, мьютекс, очереди ожидания).
- **Буферизированный vs небуферизированный:** когда блокируется отправка/получение.
- **Паника:** отправка в закрытый канал → panic.
- **Чтение из закрытого канала:** возвращает zero value и false в ok.

2.4 Интерфейсы (Interfaces)

- iface и eface: устройство интерфейса (тип, значение).
- nil-интерфейс: почему интерфейс с nil-указателем внутри не равен nil (классическая ловушка).
- **ООП в Go:** встраивание структур, реализация интерфейсов.

2.5 Структуры (Structs)

- Выравнивание памяти (alignment), размер структуры.

2.6 Паника и восстановление (Panic & Recover)

- Управляемая и неуправляемая паника.
- Как читать трассировку стека.

Подкапотка примитивов:

- sync.Mutex: битовые маски, очереди ожидания, парковка горутин.
- sync.RWMutex: отличие от Mutex.
- Каналы: hchan — буфер, мьютекс, списки ожидания.
- Атомарные операции: на уровне ассемблера — инструкция LOCK (x86).

3 2. Рантайм (Runtime)

3.1 Планировщик (GMP модель)

- **G** (Goroutine), **M** (Machine), **P** (Processor) — взаимодействие.
- **Work stealing**: как планировщик перераспределяет горутин.
- **netpoller**: обработка сетевого I/O без блокировки потоков.
- **Дисковый I/O**: блокирует горутину, но не поток (используется системный вызов).
- **Голодание горутин**: причины и как с этим бороться.
- **Модель многозадачности**: вытесняющая (preemptive) с Go 1.14; кооперативная (cooperative) — раньше.
- **GOMAXPROCS**: что это и как влияет.

3.2 Стек и куча

- **Стек горутин**: динамический, стартует с 2 КБ, растёт до 1 ГБ (64-bit).
- **Escape analysis**: как компилятор решает, где разместить переменную.
- **Куча (heap)**: общее понятие.

3.3 Сборщик мусора (GC)

- **Трёхцветный алгоритм**: маркировка и очистка.
- **Write Barrier**: зачем нужен.
- **Триггеры GC**: когда запускается сборщик.
- **Green Tea GC**: новый GC в Go 1.26 (кратко).

4 3. Конкурентность (Concurrency)

4.1 Примитивы синхронизации (sync)

- **Mutex vs RWMutex:**
 - **Mutex** — исключительная блокировка.
 - **RWMutex** — разделение читателей/писателей.
- **WaitGroup** — ожидание завершения группы горутин.
- **Once** — однократное выполнение.
- **Pool** — пул переиспользуемых объектов.
- **Map** — потокобезопасная мапа.
- **Cond** — условная синхронизация.

4.2 Атомарные операции (atomic)

- **CAS (Compare-and-Swap):** как работает.
- **Load и Store:** барьеры памяти.
- **Sequential consistency:** что это и когда применяется.
- **На уровне ассемблера:** инструкция **LOCK**.

4.3 Проблемы синхронизации

- **Deadlock (Взаимная блокировка)**
- **Livelock (Активная блокировка)**
- **Starvation (Голодание)**

4.4 Гонки данных (Data Races)

- Как детектировать: `go test -race`.
- Как избежать: синхронизация через каналы или мьютексы.

5 4. Контекст и паттерны

5.1 Пакет context

- `context.WithCancel`, `context.WithTimeout`, `context.WithDeadline` — отмена горутинов.
- Передача `context` по цепочке вызовов (первым параметром).
- Использование: таймауты, отмена запросов, передача значений (редко).

5.2 Concurrency Patterns

- **Pipeline**: последовательная обработка данных через каналы.
- **Fan-out / Fan-in**: распределение работы и сбор результатов.
- **Worker pool**: ограничение количества параллельных горутинов.
- **Применение**: эти паттерны часто спрашивают на собеседованиях (попросят написать код).

6 5. Инструменты и отладка

- **Delve (dlv)**: отладчик для Go.
- **panic и recover**: управление паникой.
- **Трассировка стека**: как читать вывод паники.
- **go test -race**: поиск гонок данных.

7 6. Рекомендации

1. **Основной фокус:** базовые типы + рантайм (планировщик, GC) + конкурентность (sync, атомики, context).
2. **Практика:** пиши код, используй `go test -race`, читай чужой код.
3. **Объясняй вслух:** каждую тему проговаривай, как будто объясняешь интервьюеру.
4. **Не закливайся на отладчиках:** в Go для отладки чаще используют трассировку паники.
5. **Изучай roadmap.sh/golang** для общей картины.

8 7. Вопросы для самопроверки

- Почему `append` может создать новый массив?
- Что будет, если отправить значение в закрытый канал?
- Как устроен слайс под капотом?
- Почему мапы небезопасны для конкурентной записи?
- Как работает планировщик Go (GMP)?
- Почему интерфейс с `nil`-указателем не равен `nil`?
- Зачем нужны `RWMutex` и `Mutex`?
- Как детектировать гонки данных?
- Что такое `work stealing` в планировщике?
- Когда использовать `sync.Map`, а когда `sync.RWMutex + map`?
- Как работают атомарные операции (`CAS`, `LOCK`)?
- Зачем нужен `context` и как им правильно пользоваться?
- В чём разница между `context.WithCancel` и `context.WithTimeout`?
- Напиши простой `pipeline` на каналах.
- Как организовать `worker pool`?